NAME:RAMCHANDRAN VS

ROLL.NO:240701420

Experiment 9: Form Validation using HTML, CSS & JavaScript

AIM:

Todesign input formsthat validate data such asemailandphone number,and display appropriateerror messages to the user using HTML, CSS, and JavaScript with the **Validator.js** library.

1.Setting Up the HTML Form

The first step is to create the skeleton of the form using plain HTML. We add two <input> fields — one for *email* and one for *phone number* — along with a submit button and placeholder tags where error messages will later appear. The Validator.js CDN link is included inside <head> so the library is available for Step 3.

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Form Validation</title>

  <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.11.0/validator.min
.js"></script>
</head>
<body>

  <h2>Contact Form</h2>

  <form id="contactForm">

    <label for="email">Email Address</label>
    <input type="text" id="email"
           placeholder="Enter your email" />
    <span id="emailError"></span>

    <label for="phone">Phone Number</label>
    <input type="text" id="phone"
           placeholder="Enter your phone number" />
    <span id="phoneError"></span>

    <button type="submit">Submit</button>

  </form>

</body>
</html>
```

OUTPUT – STEP 1

Contact Form

Email Address Phone Number Submit

Explanation: The output shows a plain, unstyled HTML form with two input boxes (Email & Phone) and a Submit button. There is no styling or validation yet — the browser uses its default font and layout. The tags for error messages are invisible at this stage because they contain no text.

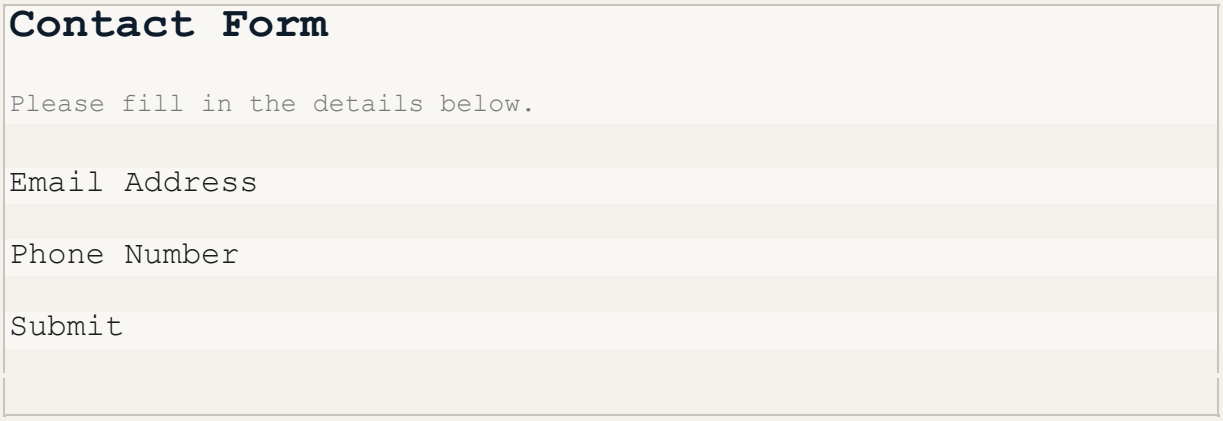
2.Styling the Form with CSS

Now we add CSS to transform the bare form into a clean, modern card-style layout. Key styling choices: a card with rounded corners and a drop shadow, uppercase labels, smooth focus transitions on inputs, and a solid-color submit button with a hover effect. We also pre-define .valid, .invalid, and .error-msg classes that JavaScript will apply in Step 3.

```
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f0f4f8;
  display: flex;
  justify-content: center;
  padding: 50px 16px;
  min-height: 100vh;
}

form {
  background: #fff;
  border-radius: 12px;
  box-shadow: 0 6px 28px rgba(0,0,0,0.12);
  padding: 30px 28px;
  width: 100%;
  max-width: 420px;
}


label {
  display: block;
  font-size: 12px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  color: #444;
  margin-bottom: 6px;
}


input {
  width: 100%;
  padding: 11px 14px;
  border: 2px solid #d8d4ce;
  border-radius: 8px;
  font-size: 14px;
  outline: none;
  background: #faf9f7;
  transition: border-color 0.2s;
  margin-bottom: 4px;
}
input:focus { border-color: #2c5f8a; background: #fff; }
```

```
input.valid    { border-color: #28a745; }
input.invalid { border-color: #dc3545;  }

.error-msg {
  display: none;
  font-size: 12px;
  color: #dc3545;
  margin-bottom: 12px;
}
.error-msg.show { display: block; }

button {
  width: 100%;
  padding: 12px;
  background: #2c5f8a;
  color: #fff;
  border: none;
  border-radius: 8px;
  font-size: 15px;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s;
  margin-top: 8px;
}
button:hover { background: #1e4468; }
```

OUTPUT – STEP 2



> **Explanation:** After applying CSS the form appears as a clean white card centred on a light grey background. Labels are small and uppercase; input fields have rounded corners and a smooth blue focus border. The submit button is styled with the primary brand colour (#2c5f8a) and darkens on hover. Error message spans are still hidden (display:none) — they will be toggled by JavaScript.

3.Adding JavaScript Validation with Validator.js

Finally we wire up the validation logic. On every input event we call validator.isEmail() and validator.isMobilePhone() to check the entered values in real time. If a value is invalid the corresponding

 becomes visible, the input border turns red, and the submit button is blocked. When both fields pass, the borders turn green and the form is allowed to submit.

JAVASCRIPT

```
const
const
const emailInput  = document.getElementById('email');
const phoneInput  = document.getElementById('phone');
const emailError  = document.getElementById('emailError');
      phoneError  = document.getElementById('phoneError');
      form        = document.getElementById('contactForm');

emailInput.addEventListener('input', () => {
  const val = emailInput.value.trim();

  if (val === '') {
    setInputState(emailInput, emailError, '', '');
  } else if (validator.isEmail(val)) {
    setInputState(emailInput, emailError, 'valid', '');
  } else {
    setInputState(emailInput, emailError,
      'invalid',
      '⚠ Please enter a valid email address.');
}
});

phoneInput.addEventListener('input', () => {
  const val = phoneInput.value.trim();

  if (val === '') {
    setInputState(phoneInput, phoneError, '', '');
  } else if (validator.isMobilePhone(val, 'any', { strictMode: false })) {
    setInputState(phoneInput, phoneError, 'valid', '');
  } else {
    setInputState(phoneInput, phoneError,
      'invalid',
      '⚠ Enter a valid phone number (e.g. +91 9876543210).');
}
});

function setInputState(input, errorEl, state, message) {
  input.classList.remove('valid', 'invalid');
  if (state) input.classList.add(state);

  errorEl.textContent = message;
  errorEl.classList.toggle('show', message !== '');
}

form.addEventListener('submit', (e) => {
  e.preventDefault();

  const emailOk = validator.isEmail(emailInput.value.trim());
  const phoneOk = validator.isMobilePhone(
      phoneInput.value.trim(), 'any', { strictMode: false });

  if (!emailOk) setInputState(emailInput, emailError,
      'invalid', '⚠ Email is required and must be valid.');

  if (!phoneOk)  setInputState(phoneInput, phoneError,
      'invalid', '⚠ Phone number is required and must be valid.');
```

```
  if (emailOk && phoneOk) {
    document.getElementById('result').className = 'success';
    document.getElementById('result').textContent =
        '✅ Form submitted successfully!';
   }
});
```

OUTPUT – STEP 3 (INVALID INPUT STATE)

Contact Form

Please fill in the details below.

Email Address

Phone Number

Submit

OUTPUT – STEP 3 (VALID INPUT + SUCCESS)

Contact Form

Try entering your details below!

Email Address

Phone Number

Submit

Explanation(ErrorState): When the user types an incomplete email like *john.doe@* or a short number like *123*, Validator.js immediately returns false. The JavaScript adds the invalid class (red border + red glow) to the input and makes the visible with a warning message. The form cannot be submitted.

Explanation (Valid State): Once a correct email (e.g. *user@example.com*) and a valid phone number (e.g. *+91 9876543210*) are entered, the borders turn green (valid class), error spans hide, and clicking Submit shows a green success banner. The live form above is interactive — try it!